

第2章 递归与分治策略

学习要点

- 理解递归的概念
- 掌握设计有效算法的分治策略
- 通过下面的范例学习分治策略设计技巧

- (1) 二分搜索技术;
- (2) 大整数乘法、**矩阵乘法**;
- (3) 棋盘覆盖;
- (4) 合并排序和快速排序;
- (5) 线性时间选择;
- (6) 最接近点对问题 ;
- (7) 循环赛日程表

预告下次课堂讨论:

矩阵乘法问题的复杂度?

分治策略总体思想

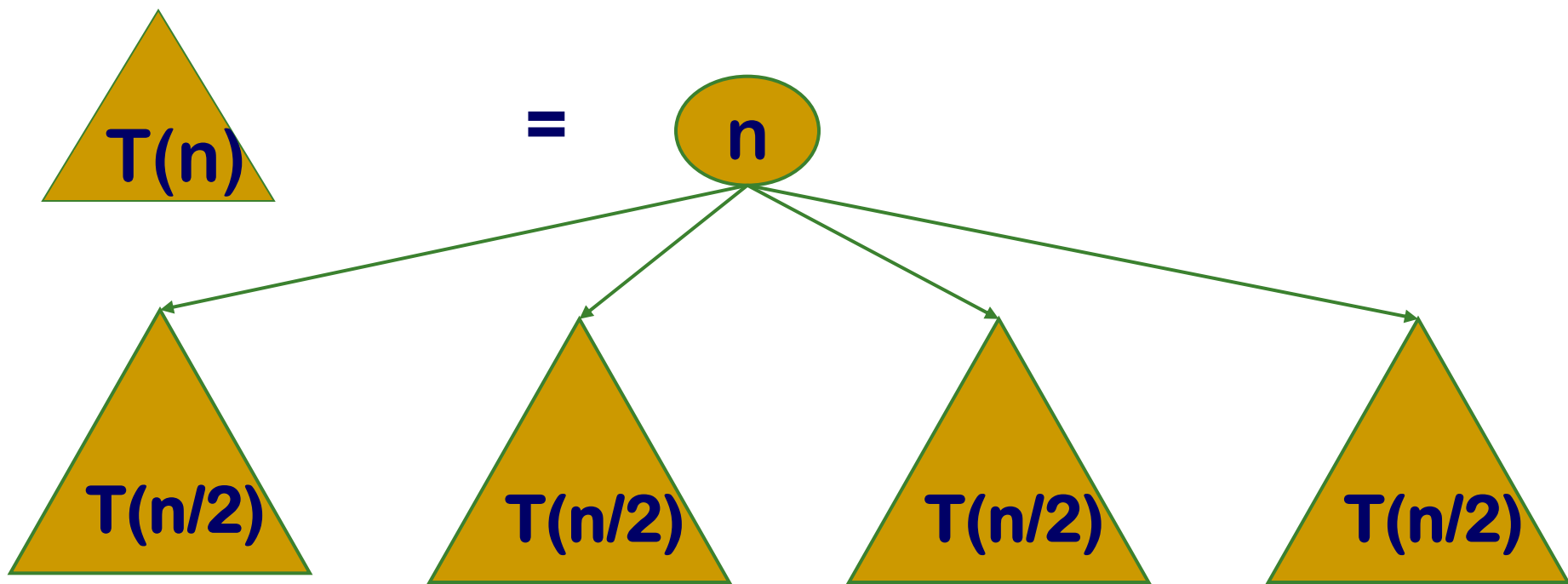
分治法 (Divide and Conquer) 是一种算法设计策略。

基本思想：

将一个难以直接解决的大问题，分解 **(Divide)** 成一些规模较小的相同问题，分别求解 **(Conquer)** 后，合并结果 **(Combine)**。

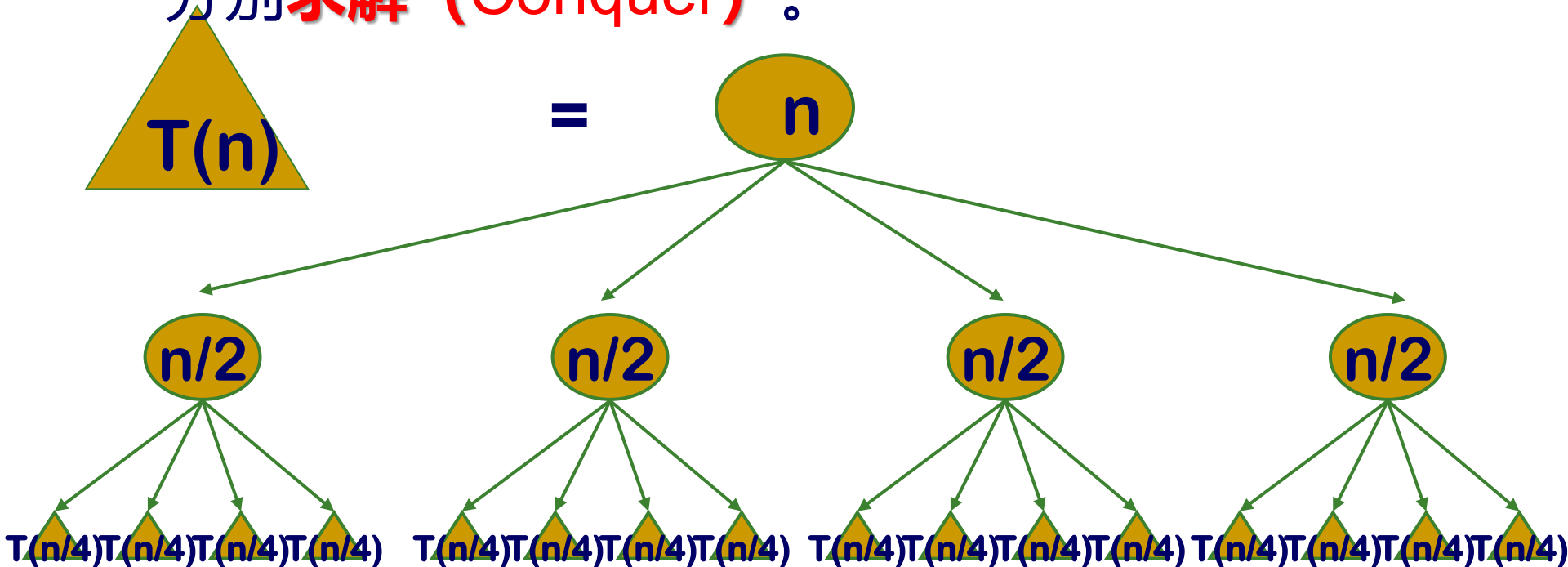
分治策略总体思想

将要求解的较大规模的问题**分解** (Divide) 成 k 个更小规模的子问题。



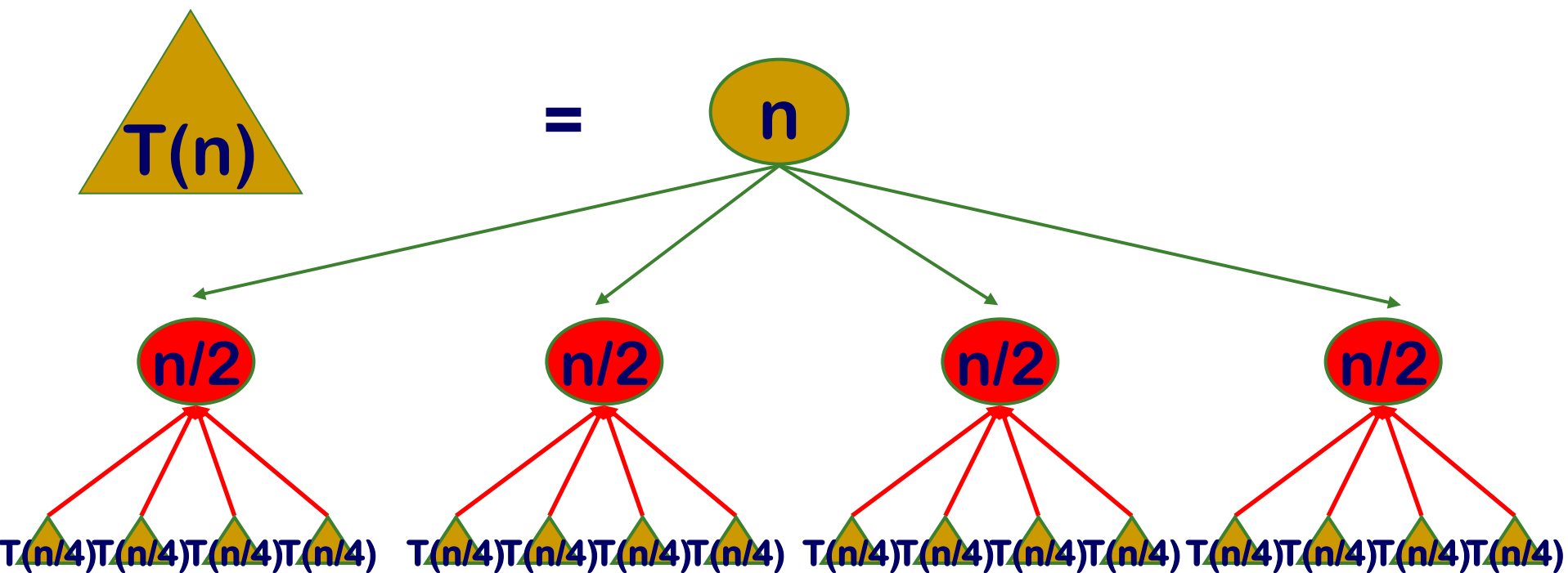
分治策略总体思想

如果子问题的规模仍然不够小，则**再分**为k个子问题，如此**递归**的进行下去，直到问题规模足够小，分别**求解** (Conquer) 。



分治策略总体思想

将求出的小规模的问题的解**合并(Combine)**为一个更大规模的问题的解，自底向上逐步求解。



2.1 递归的概念

- 分治与递归像一对孪生兄弟，经常同时应用在算法设计之中，并由此产生许多高效算法。
- **递归**是一种通过函数直接或间接调用自身来解决问题的**编程技巧**，将复杂问题逐步分解为更小的同类子问题。
- 直接或间接地调用自身的算法称为**递归算法**。
- 用函数自身给出定义的函数称为**递归函数**。

2.1 递归的概念

递归函数是计算理论中的核心概念之一，递归函数与图灵机等价，能够表示所有图灵可计算的函数。

计算载体	提出学者	计算角度
递归函数	哥德尔 Godel	数学的形式
λ - 演算 (λ - calculus)	丘奇 Church	数理逻辑的形式
图灵机	图灵 Turing	机械的形式

2.1 递归的概念

例1 阶乘函数

阶乘函数可递归地定义为：

$$n! = \begin{cases} 1 & n = 0 \\ n(n-1)! & n > 0 \end{cases}$$

边界条件

递归方程

边界条件与递归方程是递归函数的二个要素，递归函数只有具备了这两个要素，才能在有限次计算后得出结果。

2.1 递归的概念

例2 Fibonacci数列

无穷数列1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ……，称为Fibonacci数列。它可以递归地定义为：

$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

边界条件

递归方程

Fibonacci数递归计算如下：

```
int fibonacci(int n) {  
    if (n <= 1) return 1;  
    return fibonacci(n-1)+fibonacci(n-2);  
}
```

2.1 递归的概念

前2例中的函数都可以找到相应的非递归方式定义：

迭代： $n! = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$

通项公式：

$$F(n) = \frac{1}{\sqrt{5}} \left(\left(\frac{1 + \sqrt{5}}{2} \right)^{n+1} - \left(\frac{1 - \sqrt{5}}{2} \right)^{n+1} \right)$$

但是，有些函数却无法找到非递归的定义。

2.1 递归的概念

当一个函数及它的一个变量是由函数自身定义时，称这个函数是双递归函数。

例3 Ackerman函数

Ackerman函数 $A(n, m)$ 定义如下：

$$\left\{ \begin{array}{ll} A(1, 0) = 2 \\ A(0, m) = 1 & m \geq 0 \\ A(n, 0) = n + 2 & n \geq 2 \\ A(n, m) = A(A(n-1, m), m-1) & n, m \geq 1 \end{array} \right.$$

2.1 递归的概念

例3 Ackerman函数 $A(n, m)$

展开过程解析

1. 初始形式:

$$A(n, m) = A(A(n-1, m), m-1)$$

2. 第一次展开: 将最外层的 A 展开:

$$A(A(n-1, m), m-1) = A\left(A\left(A(n-1, m)-1, m-1\right), m-2\right)$$

3. 第二次展开: 继续展开内层的 A :

$$A\left(A(n-1, m)-1, m-1\right) = A\left(A\left(A(n-1, m)-2, m-1\right), m-2\right)$$

4. 第三次展开: 最终形式为:

$$A(n, m) = A\left(A\left(A\left(A(n-1, m), m-1\right), m-2\right), m-3\right)$$

2.1 递归的概念

例3 Ackerman函数

$A(n, m)$ 的自变量 m 的每一个值都定义了一个单变量函数：

$m=0$ 时, $A(n, 0)=n+2$ “加2”

$m=1$ 时, $A(n, 1)=A(A(n-1, 1), 0)=A(n-1, 1)+2$, $A(1, 1)=2$

故 $A(n, 1)=2n$ “乘2”

$m=2$ 时, $A(n, 2)=A(A(n-1, 2), 1)=2A(n-1, 2)$,
 $A(1, 2)=A(A(0, 2), 1)=A(1, 1)=2$, 故 $A(n, 2)=2^n$ 。

$m=3$ 时, 类似的可以推出

$$\underbrace{2^{2^{\cdot^{\cdot^2}}}}_n$$

$m=4$ 时, $A(n, 4)$ 的增长速度非常快, 以至于没有适当的数学式子来表示这一函数。

$A(4, 2)$ 已远超宇宙原子数

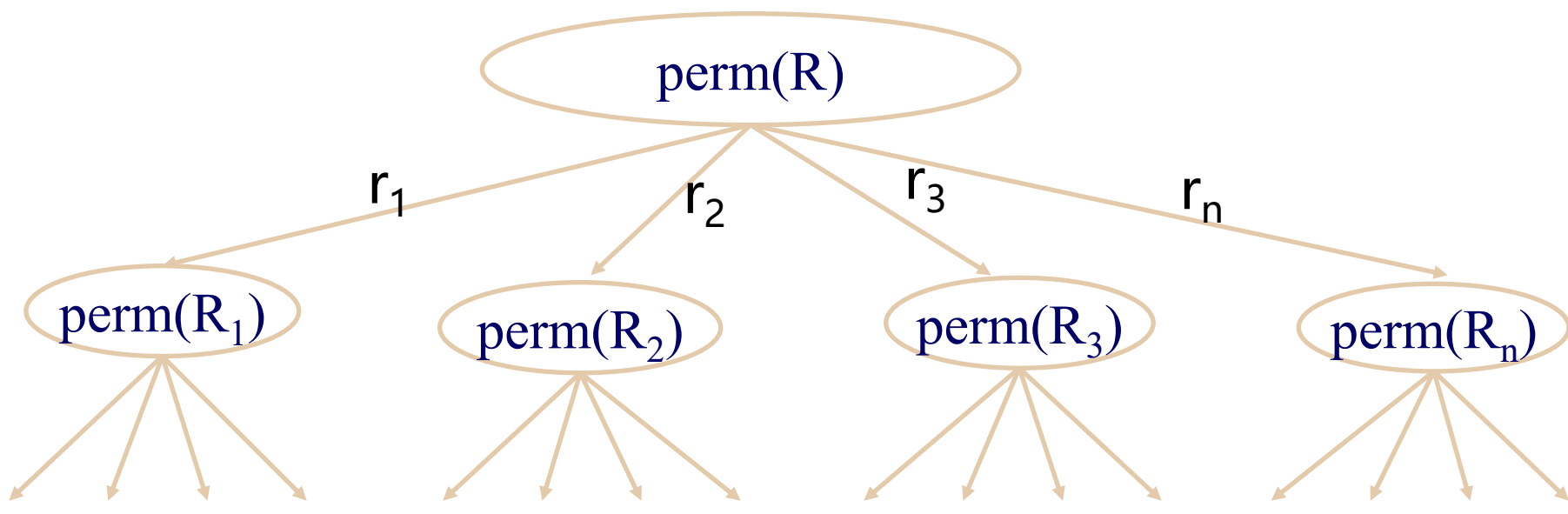
2.1 递归的概念

例4 排列问题

设计一个递归算法生成 n 个元素的全排列。

设 n 个元素的集合 $R=\{r_1, r_2, \dots, r_n\}$

R 的全排列记为 $\text{perm}(R)$, $R_i=R-\{r_i\}$ 。



2.1 递归的概念

例4 排列问题

设计一个递归算法生成n个元素的全排列。

R的全排列可递归定义如下：

当 $n=1$ 时, $\text{perm}(R)=(r)$, 其中 r 是集合 R 中唯一的元素;

当 $n>1$ 时, $\text{perm}(R)$ 由 $(r_1)\text{perm}(R_1)$, $(r_2)\text{perm}(R_2)$, ..., $(r_n)\text{perm}(R_n)$ 构成。

2.1 递归的概念

例5 整数划分问题

将正整数 n 表示成一系列正整数之和：

$$n = n_1 + n_2 + \dots + n_k,$$

其中 $n_1 \geq n_2 \geq \dots \geq n_k \geq 1$, $k \geq 1$ 。

正整数 n 的这种表示称为正整数 n 的划分。求正整数 n 的不同划分个数。

例如正整数6有如下11种不同的划分： 递归关系？

6;

5+1;

4+2, 4+1+1;

3+3, 3+2+1, 3+1+1+1;

~~2+2+2, 2+2+1+1, 2+1+1+1+1,~~

1+1+1+1+1+1。

设 $p(n)$ 为正整数 n 的划分数

增加一个自变量：
 $q(n, m)$ 表示最大加数不大于 m 的划分个数

2.1 递归的概念

例5 整数划分问题

前面的几个例子中，问题本身都具有比较明显的递归关系，因而容易用递归函数直接求解。

设 $p(n)$ 为正整数 n 的划分数，难以找到递归关系，因此考虑增加一个自变量： $q(n, m)$ 表示最大加数不大于 m 的划分个数。 建立 $q(n, m)$ 的递归关系。

$q(6, 6)$

$q(6, 3)$

$q(6, 1)$

6;

5+1;

4+2, 4+1+1;

3+3, 3+2+1, 3+1+1+1;

2+2+2, 2+2+1+1, 2+1+1+1+1;

1+1+1+1+1+1。

2.1 递归的概念

例5 整数划分问题

设 $p(n)$ 为正整数 n 的划分数，难以找到递归关系，因此考虑增加一个自变量： $q(n, m)$ 表示最大加数不大于 m 的划分个数。建立 $q(n, m)$ 的递归关系。

(1) $m=1$ (最大加数不大于1)：任何正整数 n 只有一种划分形式

$$\text{即 } n = \overbrace{1+1+\cdots+1}^n$$

$n=1$ ：1 只有一种划分形式

当 $m=1$ 或 $n=1$ 时： $q(n, m)=1$

6;

5+1;

4+2, 4+1+1;

3+3, 3+2+1, 3+1+1+1;

2+2+2, 2+2+1+1, 2+1+1+1+1;

1+1+1+1+1+1。

2.1 递归的概念

例5 整数划分问题

设 $p(n)$ 为正整数 n 的划分数，难以找到递归关系，因此考虑增加一个自变量： $q(n, m)$ 表示最大加数不大于 m 的划分个数。建立 $q(n, m)$ 的递归关系。

(2) $m > n$: 由于最大加数实际上不能大于 n $q(6, 8)$

$$q(n, m) = q(n, n)$$

6;
5+1;
4+2, 4+1+1;
3+3, 3+2+1, 3+1+1+1;
2+2+2, 2+2+1+1, 2+1+1+1+1;
1+1+1+1+1+1。

(3) $n=m$, $q(n,n)$ 根据划分中是否包含 n , 可以分为两种情况:

(a) 划分中包含 n 的情况, 只有一个即 $\{n\}$;

(b) 划分中不包含 n 的情况, 这时划分中最大的数字也一定比 n 小, 即 n 的所有 $(n-1)$ 划分。

因此 $q(n,n) = 1 + q(n,n-1)$;

6;

$$q(6,6) = 1 + q(6,5);$$

5+1;

4+2, 4+1+1;

3+3, 3+2+1, 3+1+1+1;

2+2+2, 2+2+1+1, 2+1+1+1+1;

1+1+1+1+1+1。

(4) $n > m$, 根据划分中是否包含最大值 m , 可以分为两种情况:

(a) 划分中包含 m 的情况, 即 $\{m, \{x_1, x_2, \dots, x_i\}\}$, 其中 $\{x_1, x_2, \dots, x_i\}$ 的和为 $n-m$, 个数为 $q(n-m, m)$

(b) 划分中不包含 m 的情况, 则划分中所有值都比 m 小, 即 n 的 $(m-1)$ 划分, 个数为 $q(n, m-1)$;

因此 $q(n, m) = q(n-m, m) + q(n, m-1)$;

6;

$$q(6, 4) = q(2, 4) + q(6, 3);$$

5+1;

$$q(2, 4) = q(2, 2)$$

4+2, 4+1+1;

3+3, 3+2+1, 3+1+1+1;

2+2+2, 2+2+1+1, 2+1+1+1+1;

1+1+1+1+1+1。

2.1 递归的概念

例5 整数划分问题

设 $p(n)$ 为正整数 n 的划分数，考虑增加一个自变量：将最大加数不大于 m 的划分个数记作 $q(n, m)$ 。可以建立 $q(n, m)$ 的如下递归关系。

$$q(n, m) = \begin{cases} 1 & n = 1, m = 1 \\ q(n, n) & n < m \\ 1 + q(n, n - 1) & n = m \\ q(n, m - 1) + q(n - m, m) & n > m > 1 \end{cases}$$

正整数 n 的划分数 $p(n) = q(n, n)$ 。

递归程序执行过程...

■ 函数调用机制

函数调用为了在调用函数后能正确地使用函数参数和正确地返回到调用时的位置, 必须将一些**数据存储在栈中**。这个处理过程称为函数调用机制。

■ 函数调用时,需要做下面一些工作:

- (1) 建立被调函数的**栈空间**
- (2) 保护调用函数的**运行状态和返回地址**
- (3) 保护函数传递的**参数**
- (4) 将**控制转交被调函数**

递归函数调用同样遵守函数调用机制

■递归小结

- 缺点：**递归算法的运行效率较低，无论是耗费的计算时间还是占用的存储空间都比非递归算法要多。
- 优点：**结构清晰，可读性强，而且容易用数学归纳法来证明算法的正确性，因此它为设计算法、调试程序带来很大方便。

为什么要用递归？

2.2 分治法的适用条件

分治法所能解决的问题一般具有以下几个特征：

- 该问题的**规模缩小**到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的**相同问题**；
- 利用分解出的子问题的解**可以合并**为该问题的解；
- 该问题所**分解出的各个子问题是相互独立的**，即子问题之间不包含公共的子问题。

这条特征涉及到分治法的效率，如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题，此时虽然也可用分治法，但一般用动态规划较好。

分治法的基本步骤

divide-and-conquer(P)

```
{  
  if ( | P | <= n0) adhoc(P); //解决小规模的问题  
  divide P into smaller subinstances  $P_1, P_2, \dots, P_k$ ; //分解问题  
  for (i=1, i<=k, i++)  
     $y_i = \text{divide-and-conquer}(P_i)$ ; //递归的解各子问题  
  return merge( $y_1, \dots, y_k$ ); //将各子问题的解合并为原问题的解  
}
```

问题分解方式分为两种主要类型：**递减式**分治和**等分式**分治。

2.2.1 递减式分治

例 Fibonacci 数列

$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

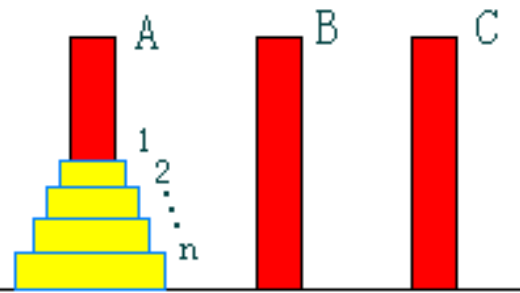
时间复杂度递推方程：

$$T(n) = T(n-1) + T(n-2) + f(n)$$

$f(n)$ 表示分解和合并的代价, 此处 $O(1)$

- 每次递归调用时, 问题规模**线性减少**
- 子问题之间**规模不均衡**

例6 Hanoi塔问题



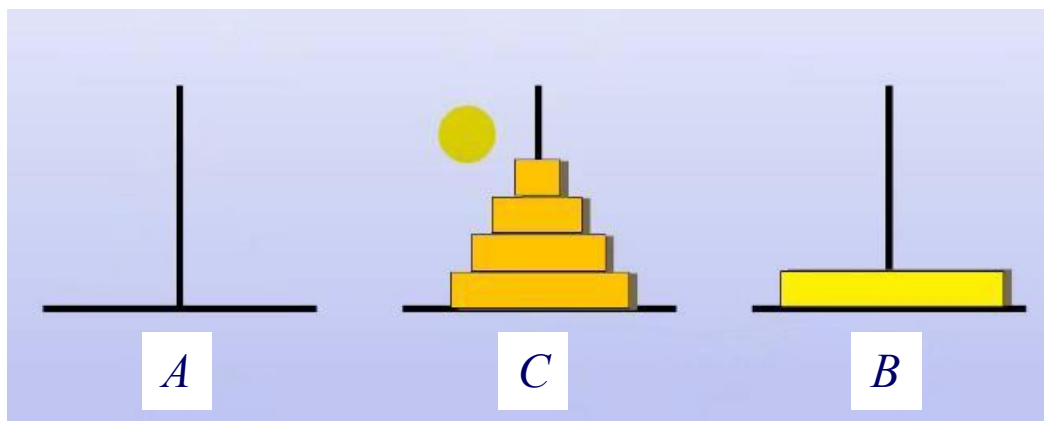
设A,B,C是3个塔座。开始时，在塔座A上有一些圆盘，这些圆盘自下而上，由大到小地叠在一起。各圆盘从小到大编号为 $1, 2, \dots, n$ ，现要求将塔座A上的 n 个圆盘移到塔座B上，并仍按同样顺序叠置。在移动圆盘时应遵守以下移动规则：

规则1：每次只能移动1个圆盘；

规则2：任何时刻都不允许将较大的圆盘压在较小的圆盘之上；

规则3：在满足移动规则1和2的前提下，可将圆盘移至A,B,C中任一塔座上。

以 $n = 5$ 个盘子为例，进行分析。
先将上面4个盘子看成一个整体，那么第一步，需要借助B柱子，将上面4个盘子放在C柱子上，然后将A柱子最底的第5个盘子放到B柱子上，如下图所示：

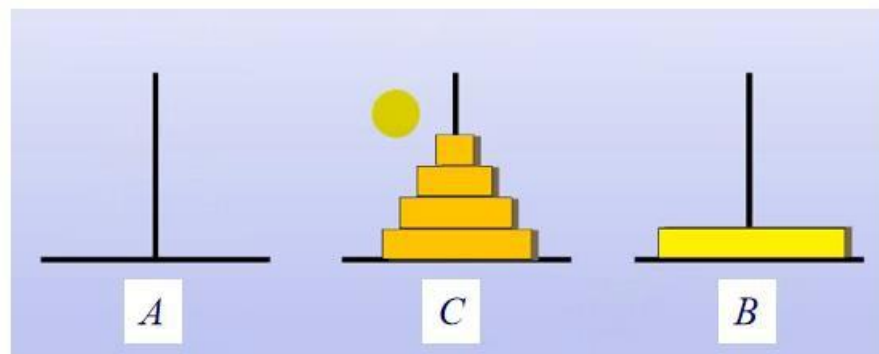


问题就依赖于C柱子上的4个盘子如何移动，也就是 $n-1$ 个盘子如何移动

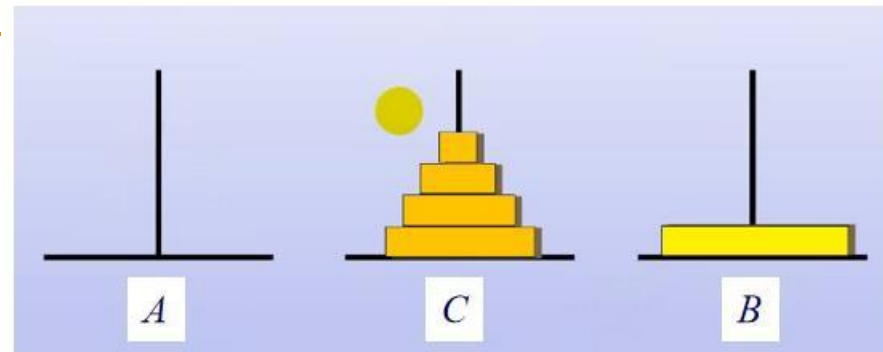
例6 Hanoi 塔问题

在问题规模较大时较难找到一般的方法，尝试用递归技术解决问题。

- 当 $n=1$ 时，问题比较简单。编号1的圆盘从塔座A直接移至塔座B。
- 当 $n > 1$ 时，利用塔座C作为辅助塔座。设法将 $n-1$ 个较小的圆盘依照移动规则从塔座A移至塔座C，然后，将剩下的最大圆盘从塔座A移至塔座B，最后，再设法将 $n-1$ 个较小的圆盘依照移动规则从塔座C移至塔座B。
- 由此可见， n 个圆盘的移动问题可分为2次 $n-1$ 个圆盘的移动问题。由此可以设计出解Hanoi塔问题的递归算法。



例6 Hanoi塔问题



```
void hanoi(int n, int a, int b, int c)  
{  
    if (n > 0)  
    {  
        hanoi(n-1, a, c, b);  
        move(a,b);  
        hanoi(n-1, c, b, a);  
    }  
}
```

函数有四个参数，影响问题规模的参数是什么？

算法复杂性分析

迭代法展开

盘子的数量 n 作为输入规模的一个指标，盘子的移动作为该算法的基本操作。

移动的次數 $M(n)$ 有下列递推等式：

$$M(n) = M(n-1) + 1 + M(n-1) = 2M(n-1) + 1, n > 1 \quad M(1) = 1,$$

逐次展开递推关系：

将递推式反复展开，直到达到基例 $M(1)$ ：

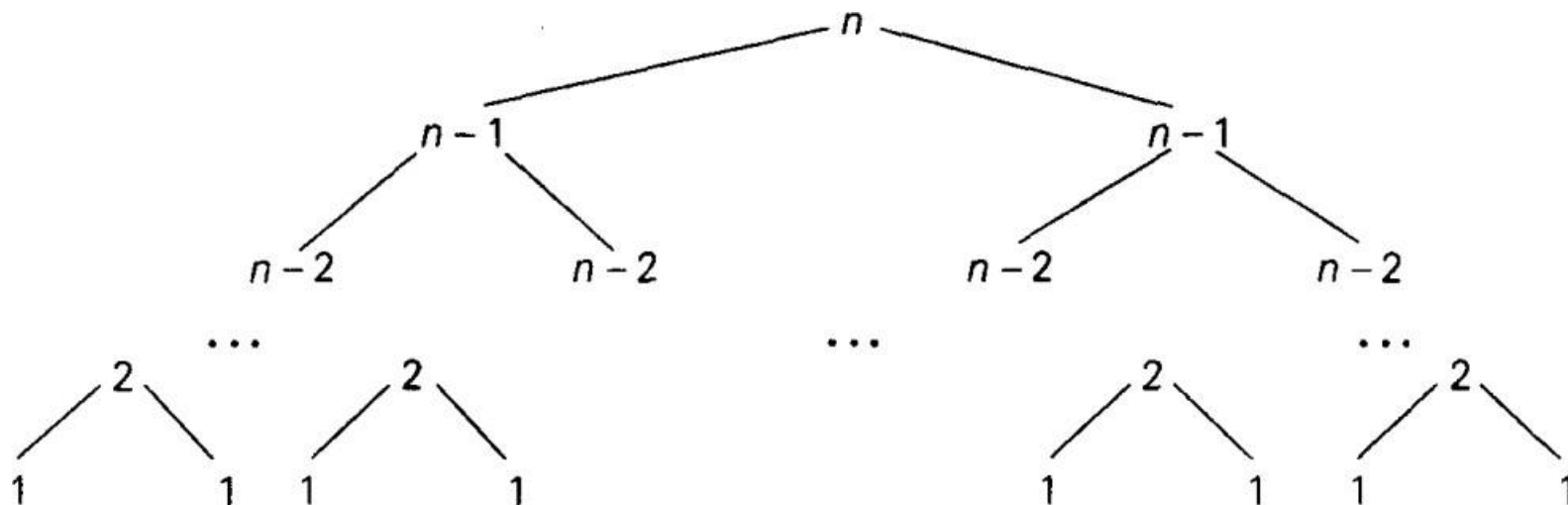
$$O(2^n)$$

$$\begin{aligned} M(n) &= 2M(n-1) + 1 \\ &= 2(2M(n-2) + 1) + 1 = 2^2M(n-2) + 2 + 1 \\ &= 2^2(2M(n-3) + 1) + 2 + 1 = 2^3M(n-3) + 2^2 + 2 + 1 \\ &\vdots \\ &= 2^{n-1}M(1) + 2^{n-2} + 2^{n-3} + \cdots + 2 + 1 = 2^n - 1. \end{aligned}$$

算法复杂性分析

递归树展开

通常一个递归算法会不止一次地调用它本身，出于分析的目的，构造一棵递归调用树是很有用的



通过计算树中节点数，可以得到汉诺塔算法所做调用全部次数：

$$M(n) = \sum_{l=0}^{n-1} 2^l = 2^n - 1$$

2.2.2 等分式分治

从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同。即将一个问题分成大小相等的 k 个子问题的处理方法是行之有效的。这种使子问题规模大致相等的做法是出自一种平衡(balancing)子问题的思想，它几乎总是比子问题规模不等的做法要好。

等分式分治的复杂性分析

若分治法将规模为 n 的问题分成 a 个规模为 n/b 的子问题。

$T(n)$ 表示该分治法解规模为 n 的问题所需的计算时间，则有

递推方程：

$$T(n) = \begin{cases} O(1) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

a : 归约后的子问题个数

n/b : 归约后子问题的规模

$f(n)$: 归约过程及组合子问题的解的工作量

迭代

$$T(n)=aT(n/b)+f(n)$$

设 $n=b^k$

$$T(n)=aT(\frac{n}{b})+f(n)$$

$$=a[aT(\frac{n}{b^2})+f(\frac{n}{b})]+f(n)$$

$$=a^2T(\frac{n}{b^2})+af(\frac{n}{b})+f(n)$$

= ...

迭代结果

$$a^{\log_b n}=n^{\log_b a}$$

$$=a^kT(\frac{n}{b^k})+a^{k-1}f(\frac{n}{b^{k-1}})+\dots+af(\frac{n}{b})+f(n)$$

$$=a^kT(1)+\sum_{j=0}^{k-1}a^j f(\frac{n}{b^j})$$

$$=\underline{c_1 n^{\log_b a}}+\sum_{j=0}^{k-1}a^j f(\frac{n}{b^j}) \quad T(1)=c_1$$

- 第一项为所有最小子问题的计算工作量
- 第二项为迭代过程归约到子问题及综合解的工作量

等分式分治的复杂性分析

求解递推方程

$$T(n) = a T(n/b) + f(n)$$

迭代法求得方程的解：

$$T(n) = n^{\log_b a} + \sum_{j=0}^{\log_b n - 1} a^j f(n / b^j)$$

主定理

$$T(n) = n^{\log_b a} + \sum_{j=0}^{\log_b n - 1} a^j f(n / b^j)$$

主定理(Master Theorem)是用于分析递归算法时间复杂度的一个重要工具，提供了分治方法带来的递归表达式的渐近复杂度分析通用方法

主定理： 设 $a \geq 1, b > 1$ 为常数， $f(n)$ 为函数， $T(n)$ 为非负整数，且

$$T(n) = aT(n/b) + f(n)$$

则有以下结果：

1. 如果 $f(n) = O(n^{\log_b a - \epsilon})$ 对某个常数 $\epsilon > 0$ 成立，则 $T(n) = \Theta(n^{\log_b a})$ 。
2. 如果 $f(n) = \Theta(n^{\log_b a} \log^k n)$ 对某个常数 $k \geq 0$ 成立，则 $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$ 。
3. 如果 $f(n) = \Omega(n^{\log_b a + \epsilon})$ 对某个常数 $\epsilon > 0$ 成立，并且对于某个常数 $d < 1$ 有 $af(\frac{n}{b}) \leq df(n)$ ，则 $T(n) = \Theta(f(n))$ 。

主定理

例题：递推方程 $T(n)=2T(n/2)+O(n\log n)$

时间复杂度分析？

1. 如果 $f(n) = O(n^{\log_b a - \epsilon})$ 对某个常数 $\epsilon > 0$ 成立, 则 $T(n) = \Theta(n^{\log_b a})$ 。
2. 如果 $f(n) = \Theta(n^{\log_b a} \log^k n)$ 对某个常数 $k \geq 0$ 成立, 则 $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$ 。
3. 如果 $f(n) = \Omega(n^{\log_b a + \epsilon})$ 对某个常数 $\epsilon > 0$ 成立, 并且对于某个常数 $d < 1$ 有 $af(\frac{n}{b}) \leq df(n)$, 则 $T(n) = \Theta(f(n))$ 。

例题：递推方程

步骤1：确定主定理参数

- $a = 2, b = 2$, 因此 $\log_b a = \log_2 2 = 1$ 。
- $f(n) = O(n \log n)$ 。

$$T(n) = 2T(n/2) + O(n \log n)$$

步骤2：比较 $f(n)$ 与 $n^{\log_b a}$

- $n^{\log_b a} = n^1 = n$ 。
- $f(n) = O(n \log n)$, 即 $f(n)$ 比 $n^{\log_b a}$ 多了一个对数因子。

步骤3：应用主定理的扩展形式

主定理的第二类扩展形式指出：

若 $f(n) = \Theta(n^{\log_b a} \log^k n)$, 则

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n).$$

- 此处 $f(n) = O(n \log n)$, 对应 $k = 1$ 。
- 因此, 时间复杂度为:

$$T(n) = \Theta(n^{\log_2 2} \log^{1+1} n) = \Theta(n \log^2 n).$$

$T(n)=2T(n/2)+O(n\log n)$

步骤1：确定主定理参数

- $a = 2, b = 2$, 因此 $\log_b a = \log_2 2 = 1$ 。
- $f(n) = O(n \log n)$ 。

课堂练习

$$T(n) = 8T\left(\frac{n}{2}\right) + 1000n^2$$

步骤2：比较 $f(n)$ 与 $n^{\log_b a}$

- $n^{\log_b a} = n^1 = n$ 。
- $f(n) = O(n \log n)$, 即 $f(n)$ 比 $n^{\log_b a}$ 多了一个对数因子。

$$T(n) = 2T\left(\frac{n}{2}\right) + 1000n$$

步骤3：应用主定理的扩展形式

主定理的第二类扩展形式指出：

若 $f(n) = \Theta(n^{\log_b a} \log^k n)$, 则

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n).$$

- 此处 $f(n) = O(n \log n)$, 对应 $k = 1$ 。
- 因此, 时间复杂度为:

$$T(n) = 2T\left(\frac{n}{4}\right) + 1000n^2$$

$$T(n) = \Theta(n^{\log_2 2} \log^{1+1} n) = \Theta(n \log^2 n).$$

学习要点

- 理解递归的概念
- 掌握设计有效算法的分治策略
- 通过下面的范例学习分治策略设计技巧
 - (1) 二分搜索技术;
 - (2) 大整数乘法, 矩阵乘法;
 - (3) 棋盘覆盖;
 - (4) 合并排序和快速排序;
 - (5) 线性时间选择;
 - (6) 最接近点对问题 ;
 - (7) 循环赛日程表